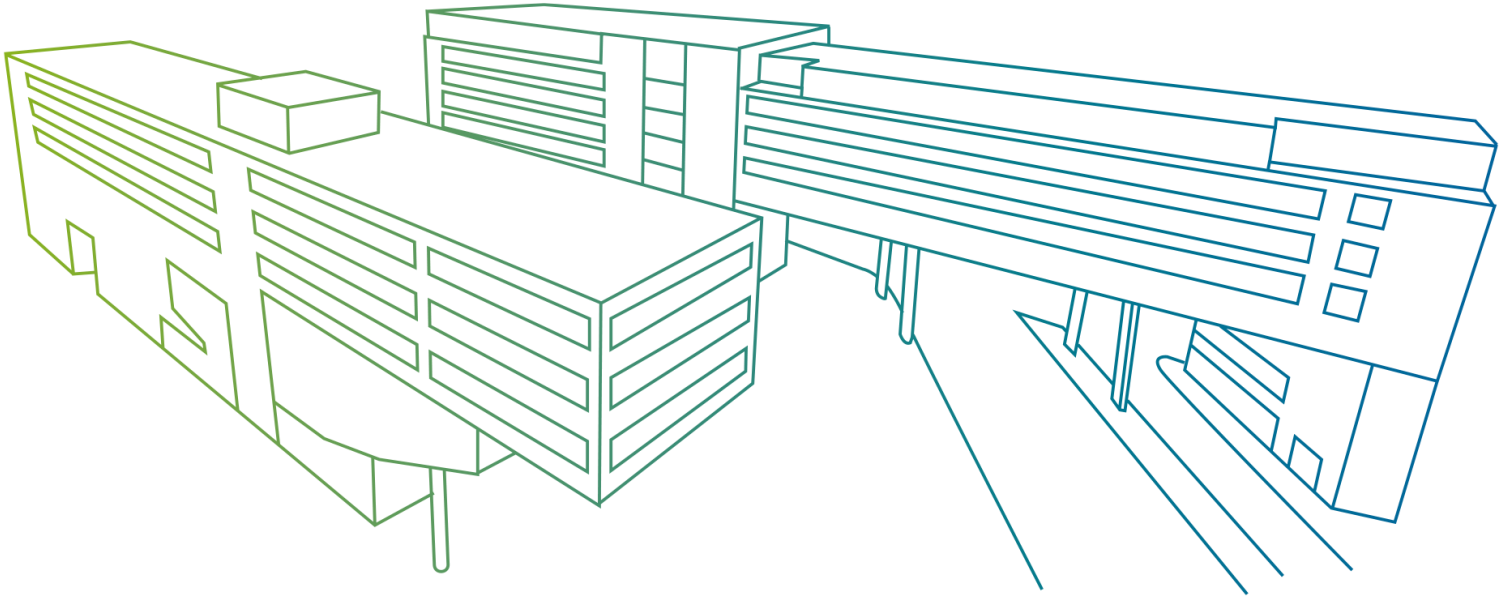


FH University of Applied Sciences TECHNIKUM WIEN

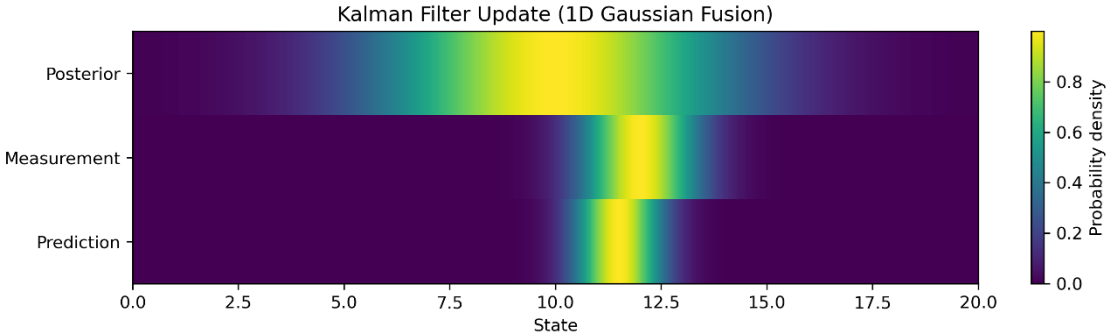
Probabilistic Robotics

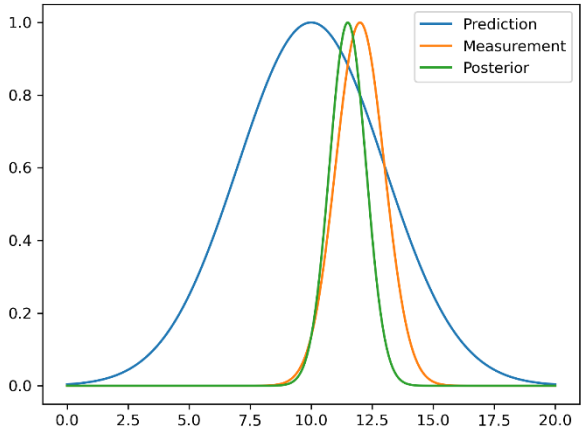
Localization & Mapping

Lucas Muster, MSc

Uncertainty as a distribution

- Belief = Probability
- We start with:
  - Our uncertainty is Gaussian
- What happens when we combine them?

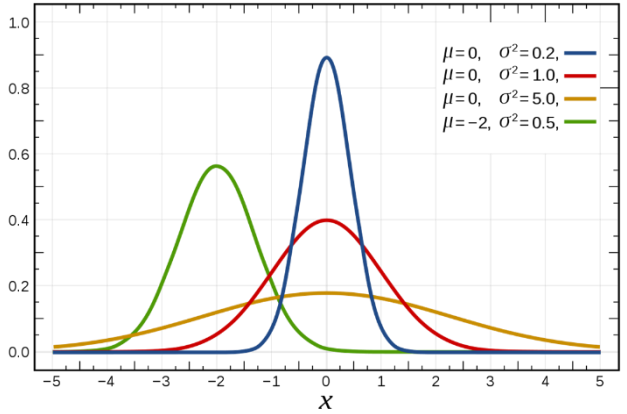


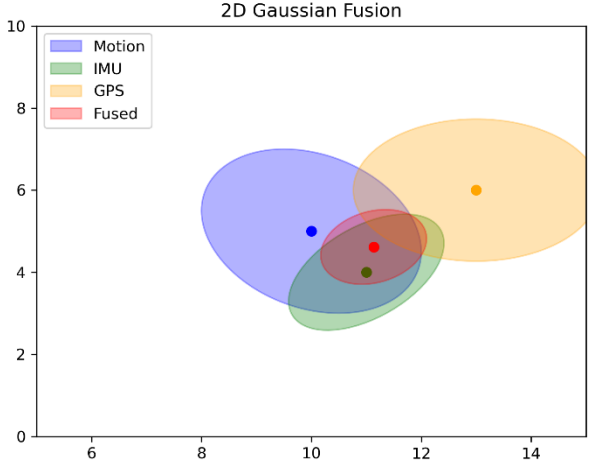


Recap: Variance = Uncertainty

- A state estimate is not a single value, but a distribution
- The variance describes how uncertain we are
- Large variance → wide Gaussian
- Small variance → narrow Gaussian

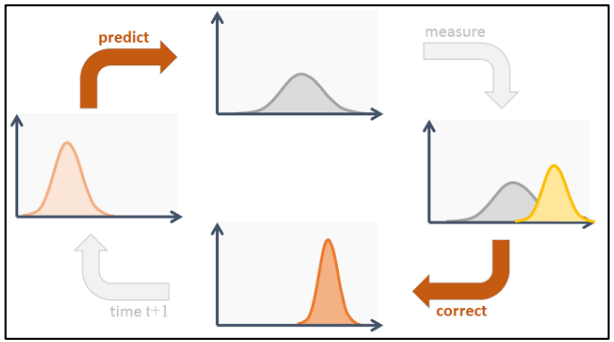
$$\Sigma = \begin{bmatrix} \sigma^2{}_x & cov_{xy} \\ cov_{yx} & \sigma^2{}_y \end{bmatrix}$$





Understanding Kalman Filter

- We discussed the motion and sensor models
- What happened to the state?
  - The expectation $\vec{\mu}_t$ and covariance after applying the motion model are calculated
  - ”Evaluation” of prediction relying on the measurement matrix C



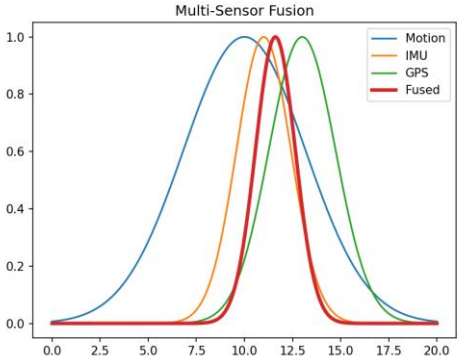
The motion is ”uncertain”: The covariance will ”grow”
The sensor observes same physical action. Based on sensor noise, we can update the prediction.

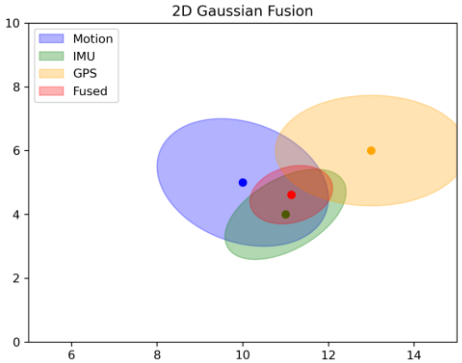
Recap: Exercise

- A robot estimates its position using three different sources:
  - Motion model (prediction)
  - IMU (short-term accurate, but noisy)
  - GPS (absolute position, but less precise)
- Each source provides an estimate with an uncertainty estimate.

- **Given:**
  - Motion model: $\mu = 10, \sigma^2 = 4$
  - IMU: $\mu = 11, \sigma^2 = 2$
  - GPS: $\mu = 13, \sigma^2 = 3$

- **Tasks**:
- Implement a function that fuses multiple Gaussian estimates.
- Compute the fused mean and variance.
- Print/Plot the result(s).





Recap: Exercise: Kalman Filter

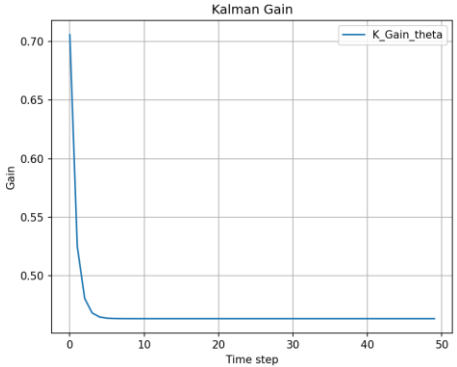
- Implement a simple Kalman Filter for robot pose estimation
- Think of any linear system with state $\vec{\mu}_t$
- Use a simple motion model and IMU measurements.
- Simulate some data (Motion and measurements)

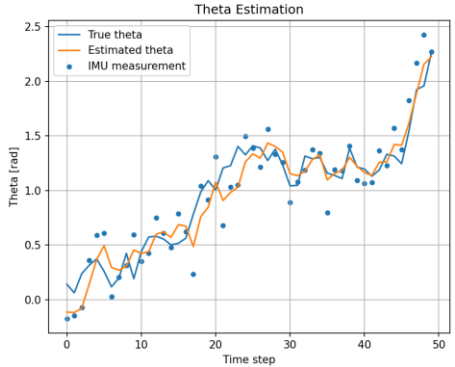
- Implement a Kalman filter
  - Prediction step (2 and 3)
  - Compute the Kalman Gain (4)
  - Implement the update step (5 and 6)

- Visualization of the true data, the estimated data and the IMU measurement
- Plot the Kalman Gain over time

1: **Algorithm Kalman_filter**$(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$**:**
2: $\bar{\mu}_t = A_t\ \mu_{t-1} + B_t\ u_t$
3: $\bar{\Sigma}_t = A_t\ \Sigma_{t-1}\ A_t^T + R_t$
4: $K_t = \bar{\Sigma}_t\ C_t^T (C_t\ \bar{\Sigma}_t\ C_t^T + Q_t)^{-1}$
5: $\mu_t = \bar{\mu}_t + K_t(z_t - C_t\ \bar{\mu}_t)$
6: $\Sigma_t = (I - K_t\ C_t)\ \bar{\Sigma}_t$
7: return μ_t, Σ_t

Figure: The Kalman filter (Thrun, 2006)





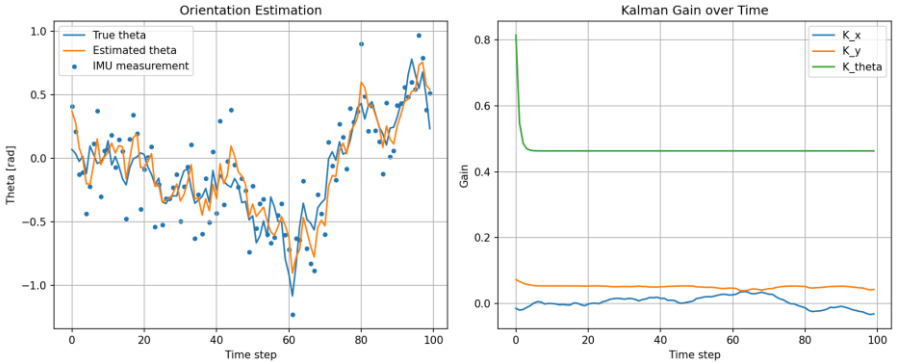
Recap: Exercise: Extended Kalman Filter

- Implement an simple EKF for robot pose estimation
- Think of any nonlinear system with state $\vec{\mu}_t$
- Use a motion model and IMU measurements.
- Simulate some data (Motion and measurements)

- Implement an Extended Kalman filter
  - Prediction step (2 and 3)
  - Compute the Kalman Gain (4)
  - Implement the update step (5 and 6)

- Visualization of the true data, the estimated data and the IMU measurement
- Plot the Kalman Gain over time

1: **Algorithm Extended_Kalman_filter**$(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$**:**
2: $\bar{\mu}_t = g(u_t, \mu_{t-1})$
3: $\bar{\Sigma}_t = G_t \, \Sigma_{t-1} \, G_t^T + R_t$
4: $K_t = \bar{\Sigma}_t \, H_t^T (H_t \, \bar{\Sigma}_t \, H_t^T + Q_t)^{-1}$
5: $\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$
6: $\Sigma_t = (I - K_t \, H_t) \, \bar{\Sigma}_t$
7: return μ_t, Σ_t



Introduction

- The problem: Drift
- Imagine a robot driving through a building using only wheel odometry.

- Small errors accumulate over time:
  - Error in $v \rightarrow$ wrong distance
  - Error in $\omega \rightarrow$ wrong angle

- These errors are integrated over time:
  - the estimated trajectory drifts away from the true trajectory
  - the uncertainty keeps growing

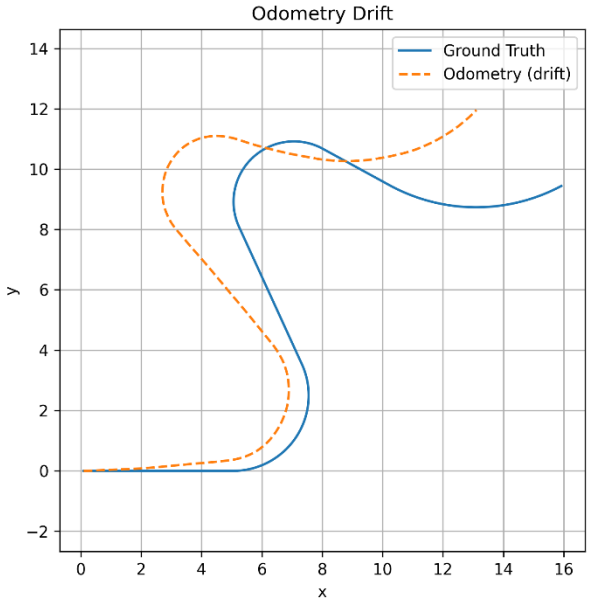
- Without external measurements, even an EKF will drift.

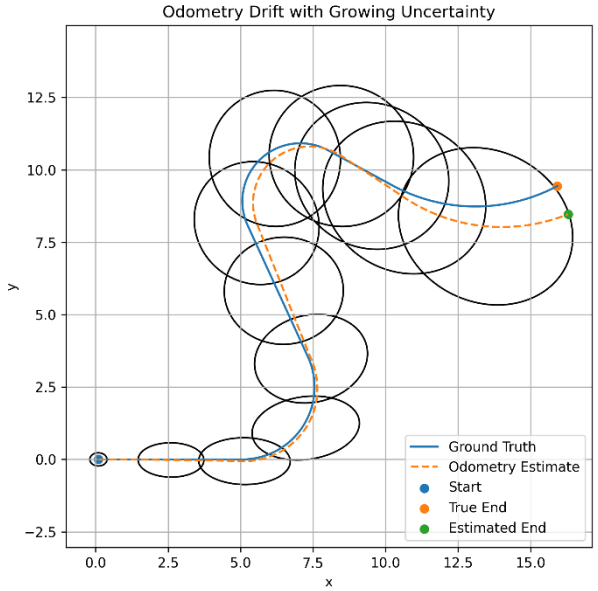
Introduction

- Till now, we fused information:
  - $\vec{u}_t$: motion command (e.g. in mobile robotics the odometry)
  - $\vec{z}_t$: the measurement
- The new state $\{\vec{\mu}_t, \Sigma_t\}$ was estimated
- However:
  - The motion is smoothed and corrected in accordance with the sensor
  - This estimation will not be true
  - Still, we drift

- Goal: to overcome this drift → but how?

Introduction

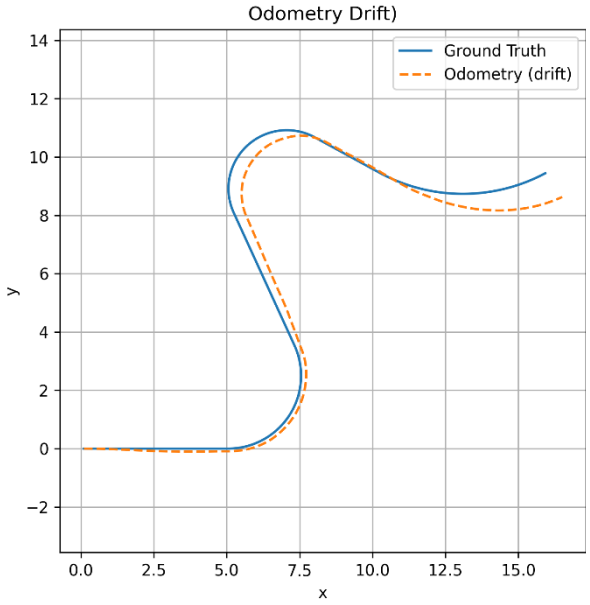


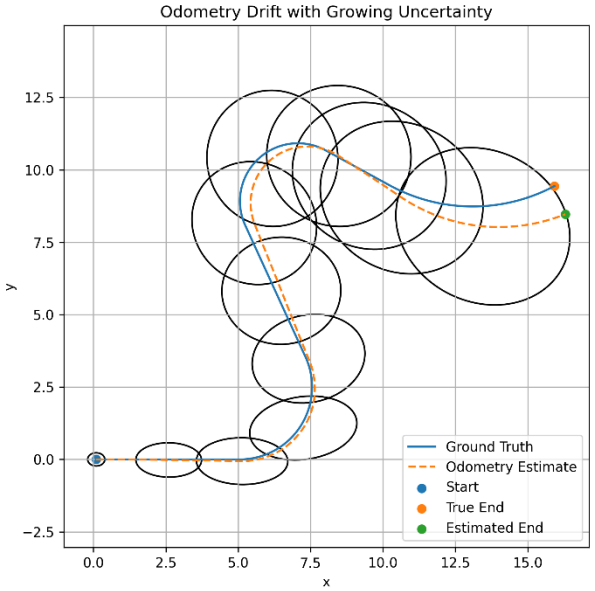


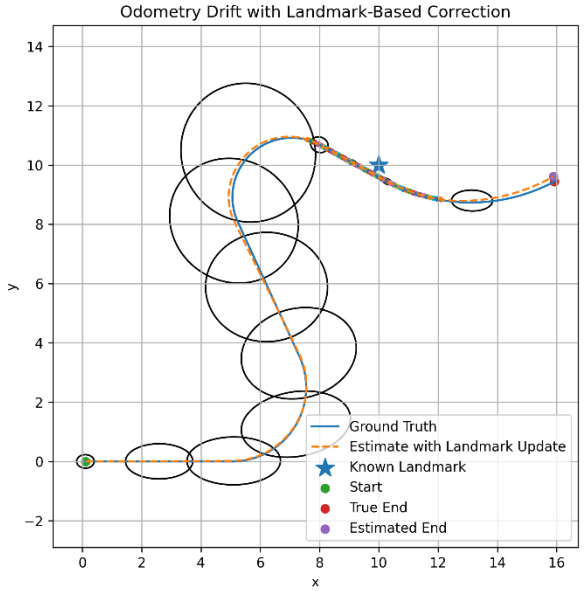
Localization with landmarks

- Therefore, we use fixed points in the world frame.
  - We try to convert the raw sensor values to more clever representations.
  - Instead of raw distances from a range finder, we detect „landmarks“.
  - Such landmarks could be:
    - A visual marker placed in the room.
    - A geometric unambiguous object (e.g. a cylinder in a empty room).
    - ...
  - The landmarks are known a-priori (stored in a „map“ before we start).
  - We adapt the EKF algorithm and use the landmarks in the correction.

EKF: Localization with landmarks

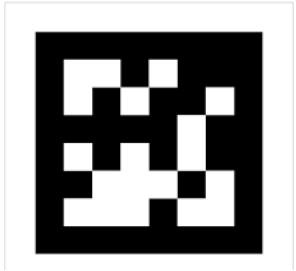






What is a landmark?

- A landmark is a known reference point in the environment
- It has a fixed position in the world
- The robot can detect and recognize it



- Examples:
  - A visual marker (e.g. AprilTag)
  - A geometric object (e.g. corner, cylinder)
  - A reflective object (LiDAR)

Landmarks and maps

- (Here) a map is a gathering of landmarks
  - 2D location in a space
  - Each landmark has a ”signature”
    - A vector with user defined information
    - Describes some properties of the landmark
  - The landmark j is described by: $m_j = (x_j \; y_j \; \vec{s}_j)$

- Thus in this chapter, a map is described by

$$m_j = \begin{bmatrix} \vec{m}_0 \\ \vdots \\ \vec{m}_n \end{bmatrix} = \begin{bmatrix} x_0 & y_0 & \vec{s}_0 \\ \vdots & \vdots & \vdots \\ x_n & y_n & \vec{s}_n \end{bmatrix}$$

Measuring a Landmark

- When the robot observes a landmark, it measures:

$$z = \begin{bmatrix} r \\ \phi \end{bmatrix}$$

r: distance to the landmark
ϕ: angle relative to the robot

- The landmark position in the map is global.
- The robot measurement is relative.

Expected Measurement

- The robot can predict what it should measure:

$$\hat{z} = h(x, m)$$

x: robot pose
m: landmark position

- The robot knows:
  - its estimated pose
  - the landmark position in the map

→We can compute the expected measurement

$$measurement\ z\ \ vs.\ prediction\ \hat{z}$$

EKF: Correspondence

- For processing, a measurement must correspond to a landmark,
- We recognise in the timeslot t the landmarks $\vec{z}_t^1, \vec{z}_t^2, \ldots, \vec{z}_t^J$

- Each measurement $\vec{z}_t^j$ corresponds to the landmark c_t^j in the map

- c_t^j allocates the measurement to a map element (a landmark).
  If c_t^j = 1, the landmark j of the measurement $\vec{z}_t^j$ belongs to map element 1.

- Correspondence vector ct

$$\vec{c}_t = (c_t^1 \ldots c_t^J)^T$$

EKF: Localization with landmarks

- Our goal: Adapt the EKF's correction

- Note: the correction can be called when a landmark is visible

- Thus we adapt the EKF:
  1. Prediction as previously: The uncertainty grows
  2. Correction: not possible all the time
  3. If we "see" a landmark: Correction is possible

→ The uncertainty will grow if no landmark is available
→ When we see a landmark the uncertainty will shrink

EKF: Localization with landmarks

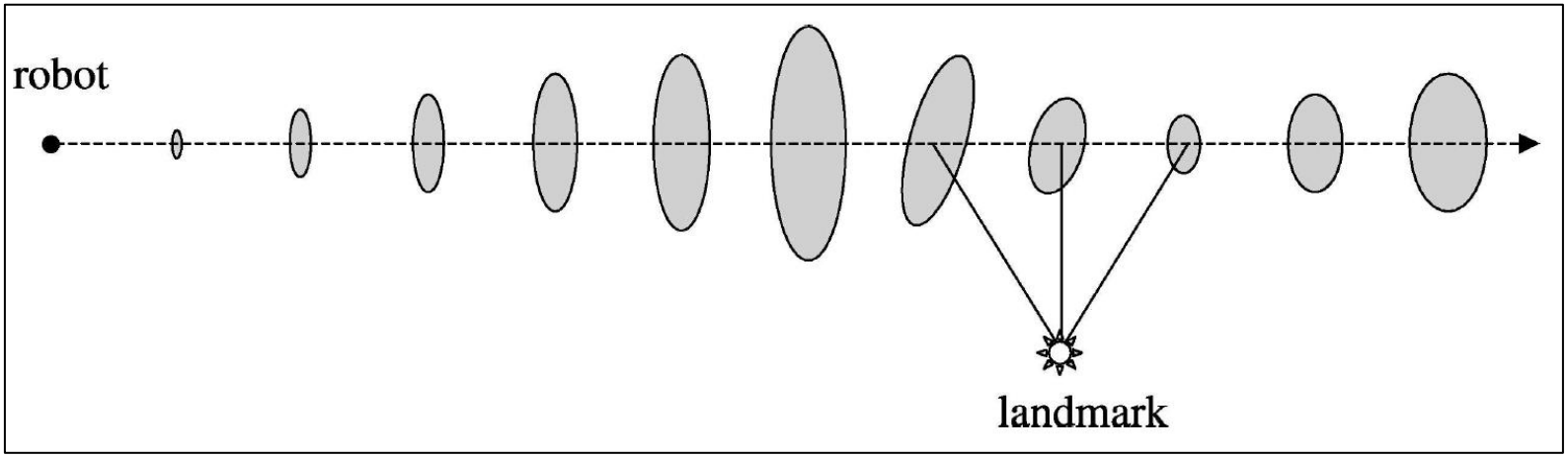
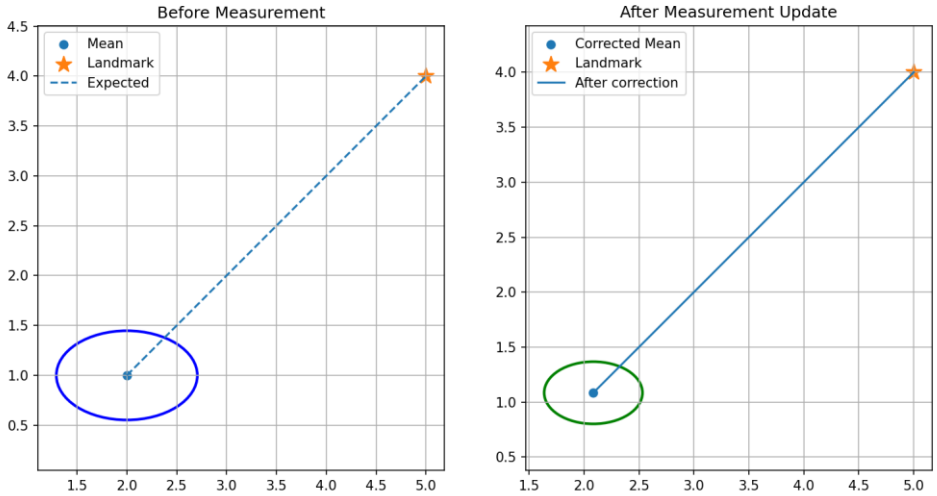


Figure: EKF Localization (Thrun, 2006)

Intuition: One Landmark Correction

- We simplify the EKF:
  1. We know:
     - robot pose (estimate)
     - landmark position (map)
  2. We can predict:
     - what we should measure
  3. We compare:
     - measurement vs prediction
  4. We correct the pose



Exercise: Landmark-based Correction

- Given:
  - Robot pose $x = (x, y, \theta)$
  - Landmark position $m = (mx, my)$

1. Compute the expected measurement:

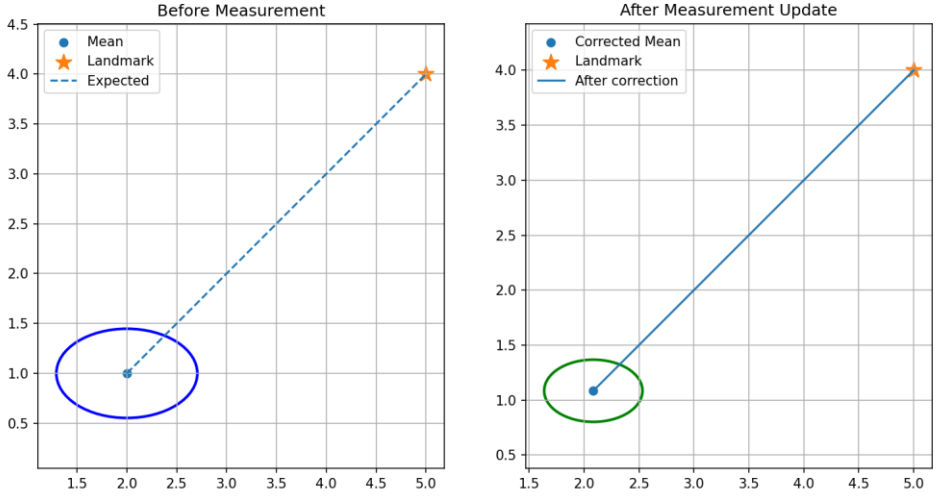
   $\hat{r}, \hat{\varphi}$

2. Assume a noisy measurement:

   r, φ

3. Compute the innovation: $e = z - \hat{z}$

4. Apply a simple correction to the pose



EKF: Adapting the Bayes Filter

- Several things need to be changed
  - The Bayes filter is missing the map
  - We must additionally conditioning on the m

1: **Algorithm Markov_localization**($bel(x_{t-1}), u_t, z_t, m$)**:**
2: *for all* x_t *do*
3: $\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1}, m)\ bel(x_{t-1})\ dx$
4: $bel(x_t) = \eta\ p(z_t \mid x_t, m)\ \overline{bel}(x_t)$
5: *endfor*
6: *return* $bel(x_t)$

Figure: Markov Localization (Thrun, 2006)

EKF: Adapting the Bayes Filter

- Now, we can adapt the structure and implement the localization in the EKF algorithm
- Steps for the implementation:
  - We assume, that the map does not influence the motion

$$p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t, m) = p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$$

  - The prediction (applying the motion model) remains the same
  - The correction processes the landmarks (if visible)
    - We process all landmarks in a loop
    - The map is in a world frame, the measurement in the sensor frame.
    - In each iteration of the loop: a individual correction
    - All other parts remains the same (Jakobi matrices, Kalman Gain)

EKF: Adapting the Bayes Filter

- We start with the (EKF) prediction
- The general algorithm is implemented for a diff. driven robot
  - g(.) is implemented
  - Based on that, G was calculated

1: **Algorithm EKF_localization_known_correspondences**$(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t, c_t, m)$**:**

2: $$\bar{\mu}_t = \mu_{t-1} + \begin{pmatrix} -\frac{v_t}{\omega_t}\sin\mu_{t-1,\theta} + \frac{v_t}{\omega_t}\sin(\mu_{t-1,\theta} + \omega_t\Delta t) \\ \frac{v_t}{\omega_t}\cos\mu_{t-1,\theta} - \frac{v_t}{\omega_t}\cos(\mu_{t-1,\theta} + \omega_t\Delta t) \\ \omega_t\Delta t \end{pmatrix}$$

3: $$G_t = \begin{pmatrix} 1 & 0 & \frac{v_t}{\omega_t}\cos\mu_{t-1,\theta} - \frac{v_t}{\omega_t}\cos(\mu_{t-1,\theta} + \omega_t\Delta t) \\ 0 & 1 & \frac{v_t}{\omega_t}\sin\mu_{t-1,\theta} - \frac{v_t}{\omega_t}\sin(\mu_{t-1,\theta} + \omega_t\Delta t) \\ 0 & 0 & 1 \end{pmatrix}$$

4: $$\bar{\Sigma}_t = G_t\,\Sigma_{t-1}\,G_t^T + R_t$$

Figure: EKF Localization (Prediction) (Thrun, 2006)

EKF: Adapting the Bayes Filter

- The landmark-based correction
  - Match map elements (L.7)
  - Convert frames (Z.8-10), calculate pseudo measurement
  - Calculate Jakobi
  - Get Kalman gain for each iteration

- Afterwards: Do correction for all landmarks

5: $Q_t = \begin{pmatrix} \sigma_r & 0 & 0 \\ 0 & \sigma_\phi & 0 \\ 0 & 0 & \sigma_s \end{pmatrix}$

6: *for all observed features* $z_t^i = (r_t^i \;\; \phi_t^i \;\; s_t^i)^T$ *do*

7: $j = c_t^i$

8: $\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} m_{j,x} - \bar{\mu}_{t,x} \\ m_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$

9: $q = \delta^T \delta$

10: $\hat{z}_t^i = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \\ m_{j,s} \end{pmatrix}$

11: $H_t^i = \frac{1}{q} \begin{pmatrix} \sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 \\ \delta_y & \delta_x & -1 \\ 0 & 0 & 0 \end{pmatrix}$

12: $K_t^i = \bar{\Sigma}_t \, H_t^{i,T} (H_t^i \, \bar{\Sigma}_t \, H_t^{i,T} + Q_t)^{-1}$

13: *endfor*

14: $\mu_t = \bar{\mu}_t + \sum_i K_t^i (z_t^i - \hat{z}_t^i)$

15: $\Sigma_t = (I - \sum_i K_t^i \, H_t^i) \, \bar{\Sigma}_t$

16: *return* μ_t, Σ_t

Figure: EKF Localization (Correction) (Thrun, 2006)

EKF: Adapting the Bayes Filter

- We can compensate drift based on global information.

- The EKF was extended with landmark-based correction.

- However, there are still problems
  - Gaussian Noise: this is still not the reality
  - Motion/Sensor model: Still an approximation
  - Single Guess: We have a single estimation of the robots pose. What happen if this is totally wrong?

Mapping

- We refer a ”map” to a set of possible representations

- A set of landmarks, where...
  - a map $\boldsymbol{m} = [\vec{m}_1^T, \ldots, \vec{m}_m^T]^T$ consists of m landmarks $\vec{m}_j$
  - each landmark is represented by $\vec{m}_j = (x_j, y_j, \vec{m}_j)$
  - each landmark has a ”signature”, a used defined vector describing any quantity

  (e.g.: color or shape information used to distinguish the landmarks)

- A set of cells describing the occupancy of a space in a frame, where...
  - the map $\boldsymbol{m} = \{m_1, \ldots, m_m\}$ is a set of cells
  - $p(m_i)$ is estimated from data
  - we want to estimate $p(\boldsymbol{m}|\vec{z}_{1:t}, \vec{x}_{1:t})$ (Note, we include the pose!)

- We focusing here on occupancy grids (Typically used in ROS - see gmapping)

Occupancy Grid Algorithm

- We now assume a known path: $\vec{x}_{1:t}$
- Since we know the path - no $\vec{u}_{1:t}$
- The measurement gives information for cell occupancy
- We use $p(m_j = 1) \coloneqq p(m_j)$ to get the probability of a occupied cell m_j
- We assume independence of the cells:

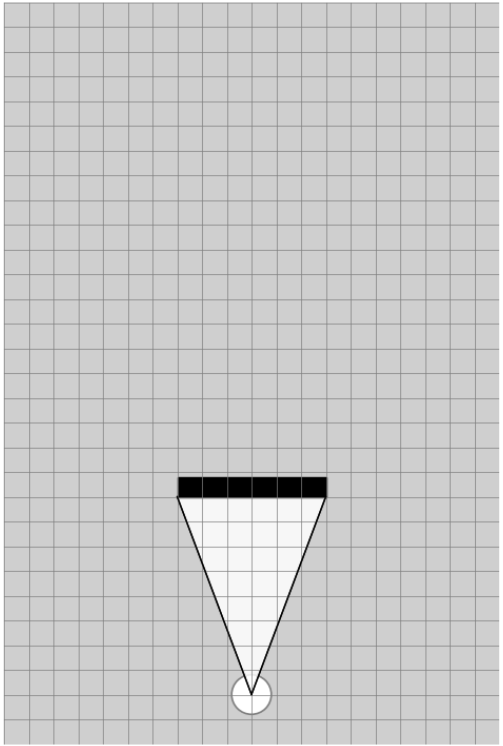
$$p(\boldsymbol{m}|\vec{z}_{1:t}, \vec{x}_{1:t}) = \prod_j p(m_j|\vec{z}_{1:t}, \vec{x}_{1:t})$$

In a huge map, evaluating $p(\boldsymbol{m}|\vec{z}_{1:t}, \vec{x}_{1:t})$ is hard - we do have many cells.

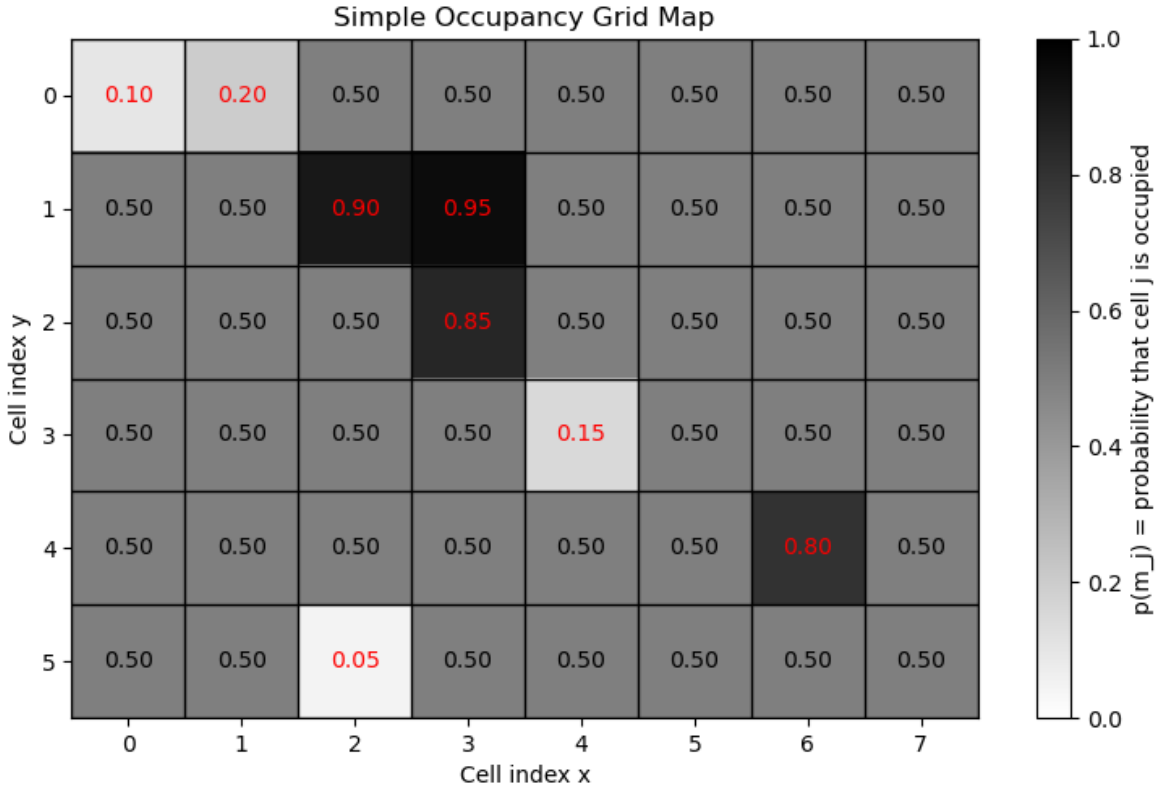
→ We use a trick: we check the sensor used for mapping (e.g.: a laser scanner) and update the cells currently in the field of view.

Occupancy Grid Algorithm

- Example of 2D scanner integration
- We analyse all valid scans (we check for z_{min} and z_{max})
- If hit is behind a cell: this cell will be ”free”
- If a hit in a cell: this cell is likely to be occupied



Occupancy Grid Algorithm



Occupancy Grid Algorithm

- We want to know all the cell information
- For each cell we want to know: How likely is the cell occupied?
- However, instead of calculating directly with the probability, we use the log-odds.
- Because updates involving probabilities are often difficult to handle.
- With log-odds, updates become simple additions rather than complicated multiplications.

$$l_{t,i} = \log \frac{p(m_j | \vec{z}_{1:t}, \vec{x}_{1:t})}{1 - p(m_j | \vec{z}_{1:t}, \vec{x}_{1:t})}$$

Occupancy Grid Algorithm

This means:

- if $l_{t,i} > 0$: The cell is more likely to be occupied
- if $l_{t,i} < 0$: The cell is more likely to be free
- if $l_{t,i} = 0$: Undecided, so typically $p = 0$.

$$l_{t,i} = \log \frac{p(m_j | \vec{z}_{1:t}, \vec{x}_{1:t})}{1 - p(m_j | \vec{z}_{1:t}, \vec{x}_{1:t})}$$

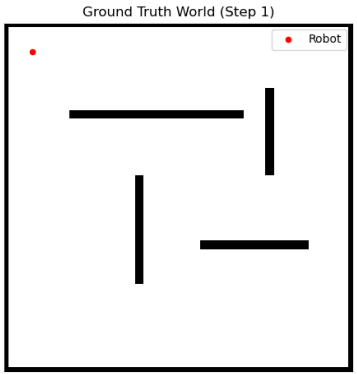
Occupancy Grid Algorithm

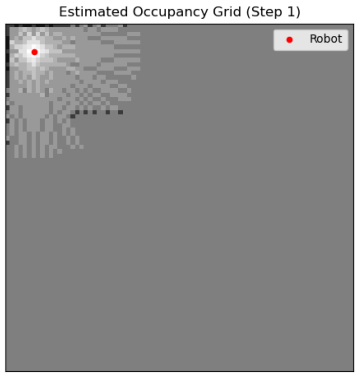
- Line 1: Function called with the prev. log odds, $\vec{x}_t$ and $\vec{z}_t$
- Line 2: This loop tries to update all cells in the map
- Line 3: Checks if current cell can be observed by the sensor
- Line 4: If yes - update the cell

1: **Algorithm occupancy_grid_mapping**$(\{l_{t-1,i}\}, x_t, z_t)$**:**
2: for all cells $\mathbf{m}_i$ do
3: if $\mathbf{m}_i$ in perceptual field of z_t then
4: $l_{t,i} = l_{t-1,i} + \textbf{inverse\_sensor\_model}(\mathbf{m}_i, x_t, z_t) - l_0$
5: else
6: $l_{t,i} = l_{t-1,i}$
7: endif
8: endfor
9: return $\{l_{t,i}\}$

Occupancy Grid Algorithm

1: **Algorithm occupancy_grid_mapping**($\{l_{t-1,i}\}, x_t, z_t$):
2: for all cells $\mathbf{m}_i$ do
3: if $\mathbf{m}_i$ in perceptual field of z_t then
4: $l_{t,i} = l_{t-1,i} + \textbf{inverse\_sensor\_model}(\mathbf{m}_i, x_t, z_t) - l_0$
5: else
6: $l_{t,i} = l_{t-1,i}$
7: endif
8: endfor
9: return $\{l_{t,i}\}$





Open problems

- We will drift somehow - when we drive cycles, it will typically not work properly (We need loop-closure methods)

- Sensors do have noise. We may need to post-process the map

- Imagine a 100 m^2 area with cells having a length of 0.01 m:

→ We need to manage 10^6 cells

- Typically, a single sensor can not distinguish very similar places and thus ambiguity occurs

SLAM: Introduction

- Simultaneous Localization and Mapping
- We do not know the map and the robot pose
- Both are uncertain and must be estimated

→Goal of SLAM: estimate robot trajectory and map simultaneously

- New approach: $p(\vec{x}_t, \vec{m} \ \ | \vec{z}_{1:t}, \vec{u}_{1:t})$
- Remember: Bayes localization $p(\vec{x}_t | \vec{z}_{1:t}, \vec{u}_{1:t}, \vec{m})$
- New questions:
  - How can we estimate the map simultaneously to the pose?
  - How can we adapt EKF localization for SLAM?

SLAM: Introduction

- SLAM tackles the problem of

$$p(\vec{x}_t, \vec{m} \;\; | \vec{z}_{1:t}, \vec{u}_{1:t}) = \int \dots \int \sum_{\vec{c}_1} \dots \sum_{\vec{c}_{t-1}} p(\vec{x}_{1:t}, \vec{m} \;\; , \vec{c}_{1:t} | \vec{z}_{1:t}, \vec{u}_{1:t}) \, d\vec{x}_1 \dots d\vec{x}_{t-1}$$

- We actually do not want to solve that now.

- In this lecture, we discuss a EKF-based simplification.
- Other methods such as the GraphSLAM are discussed in the upcoming semester.

EKF Unknown Correspondence

- In EKF and EKF localization, we now the correspondence
- Imaging an unknown correspondence vector $\vec{c}_t$
- We may estimate the correspondence

$$\hat{c}_t = \arg max_{c_t}\, p(z_t \mid c_t, m, \dots)$$

- We can use a Gaussian for that:

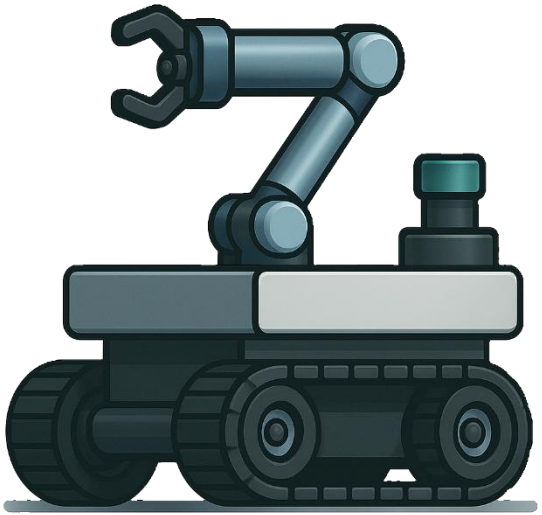
$$\arg max_k \;\det\left(2\pi S_t^k\right)^{\frac{1}{2}} \exp\left(-\frac{1}{2}\left(\vec{z}_t^{\,i} - \hat{\vec{z}}_t^{\,k}\right)^T \left(S_t^k\right)^{\frac{1}{2}} \left(\vec{z}_t^{\,i} - \hat{\vec{z}}_t^{\,k}\right)\right)$$

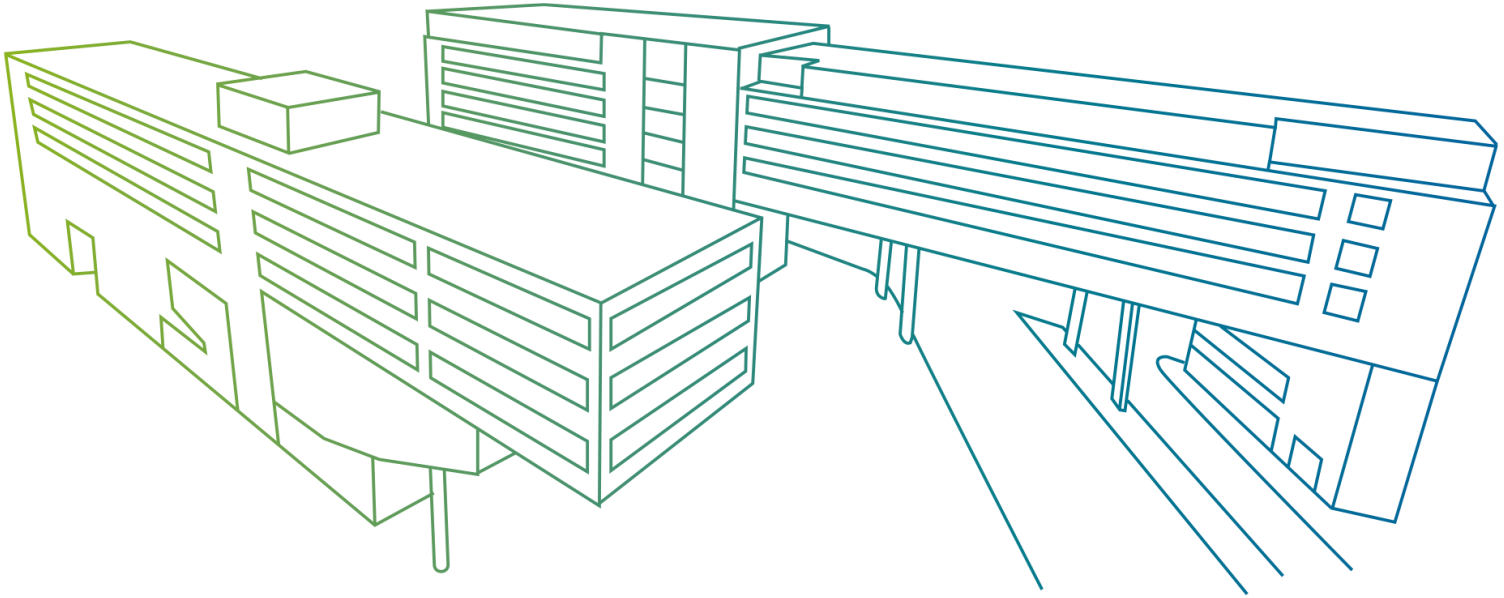
EKF Unknown Correspondence

- This can be used for a SLAM approach
- Check likelihood of landmark (See equation above)
- Decide if landmark is new or not (add landmark or use correspondence)
- If landmark is new → add it to map
- → EKF SLAM: Adapting the EKF Localization

Next steps

- Particle Filter
  - Introduction
  - Algorithm
  - Implementation
- Sensor Model
- Motion Model





FH University of Applied Sciences TECHNIKUM WIEN

Probabilistic Robotics

Localization & Mapping

Lucas Muster, MSc